

The Debian Packaging Guide

Your story: You are **hari**, a student who wanted to learn Debian packaging. Your mentor **Abhijith** taught you everything by packaging `noss` — a Perl RSS feed reader — from scratch. This guide is the distillation of that journey.

Every command here is real. Every mistake here was made. Every file here exists in your workspace at `/home/hari/junk/debian/`.

1. What Are We Actually Doing?

When you run `apt install noss`, apt downloads a `.deb` file. That `.deb` is just an `ar` archive containing three things:

```
$ ar tv noss_2.03-1_all.deb
rw-r--r-- 0/0      4  ... debian-binary    # says "2.0\n"
rw-r--r-- 0/0      ... control.tar.gz      # metadata (deps, scripts, etc.)
rw-r--r-- 0/0      ... data.tar.gz         # actual files (binaries, man pages, etc.)
```

Your job as a **maintainer** is to produce that `.deb` from upstream source code. You are the bridge between:

```
Upstream author (Samuel) → You (maintainer) → Debian archive → Users
  writes code           packages it         hosts it       apt install
```

The tools you use (`gbp`, `sbuid`, `lintian`) automate the packaging. Your real work is in 5 files under `debian/`.

2. Setting Up on a Fresh Machine

If you sat down at a new computer tomorrow, here is exactly what to do.

2.1 Install the tools

```
sudo apt install devscripts sbuild git-buildpackage quilt debmake lintian \
    debhelper dh-make wrap-and-sort debputy
```

Tool	What it does
devscripts	Meta-package: <code>debuild</code> , <code>uscan</code> , <code>dch</code> , <code>bts</code> , <code>dget</code>
sbuild	Build packages in a clean chroot (same as Debian build)
git-buildpackage	<code>gbp</code> — manages packaging workflow with git
quilt	Manage patch stacks (<code>debian/patches/</code>)
debmake	Generate <code>debian/</code> template files for new packages
lintian	Static analysis of <code>.deb</code> / <code>.changes</code> files
wrap-and-sort	Reformat <code>debian/</code> files consistently
debputy	Validate <code>debian/</code> files

2.2 Set up sbuild

sbuild builds packages in an isolated chroot so your host system stays clean.

Create `~/.config/sbuild/config.pl` (this is your exact config):

```
# ~/.config/sbuild/config.pl
$chroot_mode = 'unshare';
$distribution = 'unstable';
$build_arch_all = 1;
$build_source = 1;
$source_only_changes = 1;
$run_lintian = 1;
$lintian_opts = ['--display-info', '--verbose', '--fail-on', 'error,warning', '--info'];
$run_autopkgtest = 0;
$run_piuparts = 0;
```

Key flags explained:

Flag	What it does
<code>\$chroot_mode = 'unshare'</code>	Use Linux user namespaces (no root needed, no schroot)
<code>\$build_arch_all = 1</code>	Also build <code>Architecture: all</code> packages (like Perl)
<code>\$build_source = 1</code>	Also build the source package (<code>.dsc</code>)
<code>\$source_only_changes = 1</code>	Generate a source-only <code>.changes</code> file (for upload)
<code>\$lintian_opts = ['--fail-on', 'error,warning']</code>	Fail build if lintian finds errors OR warnings

Note: the debmake-doc suggests `~/local/sbuild/config.pl`, but `~/config/sbuild/config.pl` works identically. Use whatever you have.

2.3 Set up dquilt (for patching upstream source)

When you need to modify upstream source code (rare for Perl packages, common for C), you use quilt. The debmake-doc recommends a convenience alias:

Add to `~/bashrc` :

```
alias dquilt="quilt --quiltrc=${HOME}/.quiltrc-dpkg"
. /usr/share/bash-completion/completions/quilt
complete -F _quilt_completion $quilt_complete_opt dquilt
```

Then create `~/quiltrc-dpkg` :

```
d=.
while [ ! -d $d/debian -a `readlink -e $d` != / ]; do d=$d/..; done
if [ -d $d/debian ] && [ -z $QUILT_PATCHES ]; then
    # if in Debian packaging tree with unset $QUILT_PATCHES
    QUILT_PATCHES="debian/patches"
fi
if [ -d $d/debian ] && [ -z $QUILT_NO_DIFF_INDEX ]; then
    QUILT_NO_DIFF_INDEX=1
fi
```

This makes `dquilt new`, `dquilt add`, `dquilt refresh` store patches in `debian/patches/` instead of `patches/`.

2.4 Set up gbp (git-buildpackage)

Create `~/.gbp.conf` :

```
[DEFAULT]
builder = sbuild
pristine-tar = True
color = auto
```

- `builder = sbuild` : when `gbp buildpackage` builds, it uses `sbuild` (not `debuild`)
- `pristine-tar = True` : stores exact tarball content in git (reproducible builds)

2.5 Debian environment variables in bashrc

Set these in `~/.bashrc` :

```
export DEBEMAIL="harinarayanmr@aol.com"
export DEBFULLNAME="hari narayan"
```

- `DEBEMAIL` : your Debian email (appears in changelog)
- `DEBFULLNAME` : your name (appears in changelog)
- These are used by `dch` (the changelog tool)

2.6 Salsa account

1. Go to <https://salsa.debian.org> and try to sign up
2. Sign-ups are auto-blocked due to spam — mail `salsa-admin@debian.org`
3. Say: "I plan to maintain a package called `noss`" (or whatever your package is)
4. They will approve you
5. Add your SSH public key to your salsa profile
6. Clone repos via SSH: `git clone git@salsa.debian.org:debian/noss.git`

2.7 Email for Debian work

Debian communication happens over email (bug reports, sponsor requests, team discussions). Important rules:

- Send to `control@bugs.debian.org` in **plain text** only (Gmail has a setting for this: Settings → General → Plain text mode)
- HTML-formatted emails are rejected by Debian's bug system

2.8 git remote URL (HTTPS → SSH)

Your salsa repo will come with an HTTPS remote. Switch to SSH:

```
cd /path/to/repo
git remote set-url origin git@salsa.debian.org:debian/noss.git
```

3. The 5 Files That Are Your Job

These are the only files you need to understand deeply. Everything else under `debian/` is either auto-generated or a one-liner.

3.1 `debian/control` — The Identity Card

This file tells apt what your package is and what it needs.

Here is `noss`'s control file fully annotated:

```

Source: noss                                # Source package name (same as upstream project)
Section: net                                # Category: net, utils, perl, admin, ...
Priority: optional                           # optional/required/important/standard
Maintainer: hari narayan <harinarayanmr@aol.com> # YOU
Build-Depends:                              # What you need to BUILD the .deb
    debhelper-compat (= 13),                 # debhelper version
    perl,                                    # Perl interpreter
    libdbi-perl,                             # Perl DBI (database interface)
    libdbd-sqlite3-perl,                     # SQLite driver for DBI
    libjson-perl,                            # JSON parser
    libparallel-forkmanager-perl,            # Parallel processing
    libxml-libxml-perl,                     # XML parser
Standards-Version: 4.7.2                    # Debian policy version you followed
Homepage: https://codeberg.org/1-lsam/noss
Vcs-Git: https://salsa.debian.org/debian/noss.git # Your packaging repo
Vcs-Browser: https://salsa.debian.org/debian/noss # Web view of your repo

Package: noss                                # Binary package name (what users apt install)
Architecture: all                           # all = works on any CPU, any = compiled
Multi-Arch: foreign
Depends:                                     # What users need to RUN the program
    ${misc:Depends},                         # Auto: debhelper-generated deps
    ${perl:Depends},                         # Auto: dh_perl-generated Perl deps
    dialog,                                 # Needed by nossui (TUI frontend)
    sensible-utils,                         # Needed by nossui (sensible-editor)
    perl,                                    # Perl interpreter
    libdbi-perl,                             # (these match Build-Depends)
    libdbd-sqlite3-perl,
    libjson-perl,
    libparallel-forkmanager-perl,
    libxml-libxml-perl,
    curl,                                    # Feed downloader
    lynx,                                    # HTML-to-text converter
    sqlite3,                                # Database engine
Description: command-line program for aggregating and reading RSS/Atom
    feeds.
    noss is a command-line program for aggregating and reading RSS/Atom
    feeds. ...

```

Key concepts:

- **Build-Depends vs Depends:** Build-Depends are installed *only in the build chroot*. Depends are required by the *end user*. If a user needs it to run, it goes in Depends. If you only need it during `make`, it goes in Build-Depends.
- `${perl:Depends}`: `dh_perl` scans your Perl scripts, finds all `use Module` statements, and converts them to Debian package names. You don't list Perl modules manually in Depends if they're detected automatically. But you *do* need them in Build-Depends (so the build chroot has them).

- `${misc:Depends}` : debhelper adds its own deps here (like `debconf` if your package uses it, or `dpkg` for certain features).
- **Architecture all vs any**: Perl scripts are `Architecture: all` (same .deb for all CPUs). C programs are `Architecture: any` (compiled per CPU).
- **Description rules**: First line is the "short description" (one sentence, no leading space). Continuation lines start with exactly one space. No tabs. Wrapped at ~70 chars. Wrong alignment → lintian error.

3.2 `debian/changelog` — The Release History

Every release needs a changelog entry. This is the format:

```
noss (2.03-1) UNRELEASED; urgency=low

* new upstream release.
* added sensible-utils to Depends.

-- hari narayan <harinarayanmr@aol.com>  Fri, 15 May 2026 04:02:02 +0530
```

The version number decoded:

```
noss (2.03-1)
      ^  ^
      |   Debian revision (starts at 1, incremented for packaging fixes)
      |   Upstream version (from upstream author)
```

- If you fix a packaging bug without a new upstream release: `2.03-2` , `2.03-3` , etc.
- If upstream releases `2.04` : `2.04-1`

Distribution field: - `UNRELEASED` = development mode. sbuild works, lintian warns, but it won't reach the archive. Keep this until you're ready. - `unstable` = ready for upload to Debian unstable. - `stable` , `testing` , `experimental` = other suites.

Changelog rules from your experience: - All lowercase: maintainer name in lowercase, entries in lowercase - Email must be `harinarayanmr@aol.com` (the original, not gmail) - Date format: `Day, DD Mon YYYY HH:MM:SS +TZ` (e.g. `Fri, 15 May 2026 04:02:02 +0530`) - Each entry describes *what changed in this Debian version*

Multi-entry changelog (after an update):

```
noss (2.03-1) UNRELEASED; urgency=low

* new upstream release.

-- hari narayan <harinarayanmr@aol.com> Fri, 15 May 2026 04:02:02 +0530

noss (1.10-1) unstable; urgency=low

* Initial release. (Closes: #1111381)

-- hari narayan <harinarayanmr@aol.com> Sat, 06 Dec 2025 18:13:42 +0530
```

Older entries stay below, newest on top.

3.3 `debian/copyright` — The License Document

DEP-5 format. Tells users and automated tools who wrote the code and under what license.

```
Format: https://www.debian.org/doc/packaging-manuals/copyright-format/1.0/
Upstream-Name: noss
Upstream-Contact: <samyoung12788@gmail.com>
Source: https://codeberg.org/1-lsam/noss

Files: *
Copyright: 2025 Samuel Young <samyoung12788@gmail.com>
License: GPL-3.0+

Files: debian/*
Copyright: 2025 hari narayan <harinarayanmr@aol.com>
License: GPL-3.0+

License: GPL-3.0+
This program is free software: you can redistribute it and/or modify
it under the terms of the GNU General Public License as published by
the Free Software Foundation, either version 3 of the License, or
(at your option) any later version.
...
```

Two stanzas minimum: 1. `Files: *` — covers all upstream code 2. `Files: debian/*` — covers your packaging work

If upstream has files under different licenses, add more stanzas:


```
Files: contrib/*
Copyright: 2025 Samuel Young
License: GPL-3.0+

Files: vendor/*.c
Copyright: 2022 Some Other Person
License: BSD-2-Clause
```

When upstream releases a new version, check if they added any files with different licenses. For `noss v2.03`, they added a screenshot (`img/noss-list-screenshot.png`) — but it's under the same `GPL-3.0+`, so no change needed.

3.4 `debian/rules` — The Build Recipe

For most Perl packages, this is the entire file:

```
#!/usr/bin/make -f

%:
    dh $@
```

That's it. `dh $@` runs the entire debhelper sequence: `dh_auto_configure` → `dh_auto_build` → `dh_auto_test` → `dh_install` → `dh_installdocs` → `dh_strip` → `dh_compress` → `dh_fixperms` → `dh_gencontrol` → `dh_md5sums` → `dh_builddeb`

For Perl packages specifically: - `dh auto_configure` detects `Makefile.PL` and runs `perl Makefile.PL` - `dh auto_build` runs `make` - `dh auto_test` runs `make test` - `dh_perl` scans for Perl module dependencies and generates `${perl:Depends}` - `dh_install` puts files in the right places (including `bin/noss` , `bin/nossui`)

When you would customize rules: - Passing flags to compiler:

`DEB_CFLAGS_MAINT_APPEND` - Skipping tests: `dh_auto_test` override - Adding install steps: override `dh_install` or `dh_auto_install`

For `noss`, the defaults are perfect. Don't touch `debian/rules` unless needed.

3.5 `debian/watch` — The Upstream Version Tracker

This file tells `uscan` how to check for new upstream releases.

```
version=4
https://codeberg.org/1-lsam/noss/tags .*/noss/archive/(\d[.\d]+)\.tar\.gz
```

- `version=4` : use version 4 format (the current standard)
- The URL is a page that lists download links (Codeberg's tags page)
- The pattern extracts the version number from each link

What uscan does with this:

1. Downloads `https://codeberg.org/1-lsam/noss/tags`
2. Scans for links matching `.*\/noss/archive/(\d[.\d]+)\.tar\.gz`
3. Extracts versions from capture group: `1.10` , `2.03` , `2.04` , etc.
4. Compares with your current version (from `debian/changelog`)
5. Reports if a newer version exists

Common mistakes: - `Version: 5` (wrong — this is v1 format syntax): use `version=4` (all lowercase) - Old v1 format with `Source:` and `Matching-Pattern:` keywords: v4 format uses a single line with URL + pattern

4. The First Package — Story of noss

This is the full workflow Abhijith walked you through, from nothing to a published package.

4.1 Before You Package: The ITP Bug

The first step when packaging something new: tell the world you're working on it.

1. Check if someone already filed an RFP (Request For Package): `https://bugs.debian.org/cgi-bin/pkgreport.cgi?pkg=noss`
2. If an RFP exists, retile it to ITP (Intent To Package): `To: control@bugs.debian.org`
`Subject: retile 1111381 ITP: noss -- Command-line RSS/Atom feed reader and aggregator`
`owner 1111381 harinarayanmr@aol.com` Send in **plain text**.
3. This prevents duplicate work. Other maintainers know you're handling it.

4.2 Clone the Salsa Repo

Abhijith created the repo at `salsa.debian.org/debian/noss` for you.

```
git clone git@salsa.debian.org:debian/noss.git
cd noss
```

4.3 Import the Upstream Source

Download the upstream tarball:

```
curl -L -o ../noss-1.10.tar.gz https://codeberg.org/1-lsam/noss/archive/1.10.tar.gz
```

Then import it with gbp:

```
gbp import-orig --verbose --pristine-tar ../noss-1.10.tar.gz
```

What this command does:

1. Extracts the tarball to a temporary directory
2. Creates a commit on the `upstream` branch with the new source
3. Creates a tag `upstream/1.10`
4. Merges the upstream branch into `master`
5. Preserves your `debian/` directory
6. Stores a pristine-tar delta (so you can recreate the exact tarball later)

Important: gbp renames your source directory to match the version:

```
test/noss-1.10/ → test/noss-2.03/
```

This happens every time you import a new version. Always check `pwd` after import-orig.

If the version isn't detected automatically:

```
gbp import-orig --verbose --pristine-tar --upstream-version=1.10 ../noss-1.10.tar.gz
```

4.4 Edit the debian/ Files

After import, you need to edit the 5 files (chapter 3) to match your package.

Your first-time edits for noss: - `debian/control` — add Build-Depends (`libdbi-perl`, `libdbd-sqlite3-perl`, etc.), add Depends (`dialog`, `curl`, `lynx`, `sqlite3`, etc.), write Description - `debian/copyright` — two stanzas (upstream + debian) - `debian/changelog` — set version 1.10-1 UNRELEASED, write entry - `debian/watch` — version 4 format with Codeberg URL

How to find what deps you need:

The upstream `Makefile.PL` lists Perl module dependencies in `PREREQ_PM`:

```
PREREQ_PM => {
    'DBI' => '1.614',
    'DBD::SQLite' => '0',
    'JSON' => '0',
    'Parallel::ForkManager' => '0.7.6',
    'XML::LibXML' => '1.70',
    ...
},
```

Each Perl module maps to a Debian package. The convention:

```
Perl module:      DBI
Debian package:   libdbi-perl

Perl module:      XML::LibXML
Debian package:   libxml-libxml-perl

Perl module:      Parallel::ForkManager
Debian package:   libparallel-forkmanager-perl
```

The pattern: `lib<name>-perl` with `::` replaced by `-` and lowercase.

To verify a package exists:

```
apt-cache search libdbi-perl
```

Also check for non-Perl deps: `curl` (binary for downloads), `lynx` (HTML rendering), `sqlite3` (database), `dialog` (TUI frontend). These are found in the upstream README or by reading the scripts.

4.5 Build with sbuild

From inside the source directory:

```
sbuid -sA --no-clean-source --dist=unstable
```

Flag	Meaning
<code>-s</code>	Build source package too
<code>-A</code>	Build Architecture: all packages
<code>--no-clean-source</code>	Don't clean the source tree first (needed when rebuilding)
<code>--dist=unstable</code>	Build for unstable (matches your sbuild config)

What happens during a build (automated by sbuild):

1. Create a clean chroot (if cached chroot exists, reuse it)
2. Install Build-Depends inside the chroot
3. Copy source into chroot
4. Run `dpkg-buildpackage` which calls `debian/rules` targets: `- clean` → `dh clean` - `build` → `dh build` → `dh_auto_configure` → `dh_auto_build` - `build-arch` / `build-indep` → architecture-specific builds - `binary` → `dh binary` → `dh_install`, `dh_perl`, `dh_gencontrol`, etc. - `binary-arch` / `binary-indep` → architecture-specific binary
5. Run lintian on the result
6. Copy artifacts (`.deb` , `.changes` , `.buildinfo`) to parent directory

Reading the build summary:

```
Build Architecture: amd64
Build Type: full
Build-Space: 1672
Build-Time: 9
Package-Time: 96
Status: successful
Lintian: fail          # meaning lintian found errors – check the log
Piuparts: fail         # known issue, doesn't affect the .deb
Autopkgtest: pass     # tests passed
```

- `Status: successful` — the `.deb` was produced correctly
- `Lintian: fail` — lintian found something. Examine the log for `E:` and `W:`
- `Piuparts: fail` — piuparts checks install/uninstall; often fails for beginners

4.6 Run Lintian

```
lintian -i ../noss_2.03-1_amd64.changes
```

The `-i` flag gives detailed explanations of each issue. For example:

```
E: noss: missing-depends-on-sensible-utils sensible-editor [usr/bin/nossui]
```

Meaning: `nossui` uses `sensible-editor` but `sensible-utils` is not in Depends. Fix: add `sensible-utils` to Depends in control and rebuild.

Common issues you'll see:

Issue	Severity	What it means	Fix
<code>unreleased-changes</code>	Error	Changelog says UNRELEASED	Change to <code>unstable</code> when ready
<code>missing-depends-on-*</code>	Error	Missing runtime dependency	Add to Depends in control
<code>initial-upload-closes-no-bugs</code>	Warning	No ITP bug closed	Add <code>(Closes: #NNN)</code> in changelog
<code>wrong-manual-section 1 != 1p</code>	Warning	Perl man page in wrong section	Upstream issue, add lintian override or fix upstream
<code>no-nmu-in-changelog</code>	Warning	Changelog format not NMU	Ignore if this is your own package

4.7 Push to Salsa

After a successful build:

```
git push --all      # push all branches (master, upstream, pristine-tar)
git push --tags     # push tags (upstream/1.10, upstream/2.03)
```

Tags must be pushed separately! `git push` alone does not push tags. If you forget, Abhijith can't pull your upstream source.

4.8 The Upload (Sponsor Does This)

For your first upload, your sponsor (Abhijith) handles this:

1. They pull your salsa repo
2. Build locally to verify
3. Run `dput` to upload the `.changes` file to the Debian archive

4. The package goes through NEW queue review
5. It appears in the archive 1-3 days later

Your DDPO (Debian Developer Portfolio) page shows your contributions: <https://qa.debian.org/developer.php?login=harinarayanmr%40aol.com>

5. The Update Workflow (What You'll Do Most)

This is the most important chapter. You'll do this every time upstream releases a new version.

5.1 Check for New Versions

```
cd test/noss-<current-version>
uscan --verbose --report
```

This uses `debian/watch` to check upstream. Output looks like:

```
Newest version of noss on remote site is 2.03, local version is 1.10
=> Newer package available from:
    => https://codeberg.org/1-lsam/noss/archive/2.03.tar.gz
```

5.2 Download the New Tarball

If uscan works (and your watch file is correct):

```
uscan --download-version 2.03
```

If uscan times out or fails (Codeberg can be slow):

```
curl -L -o ../noss-2.03.tar.gz https://codeberg.org/1-lsam/noss/archive/2.03.tar.gz
```

The tarball goes to the parent directory (root of your workspace).

5.3 Import with gbp

```
gbp import-orig --verbose --pristine-tar --upstream-version=2.03 ../noss-2.03.tar.gz
```

What you'll see:

```
gbp:info: Importing '../noss-2.03.tar.gz' to branch 'upstream'...
gbp:info: Source package is noss
gbp:info: Upstream version is 2.03
gbp:info: Replacing upstream source on 'master'
gbp:info: Successfully imported version 2.03 of ../noss_2.03.orig.tar.gz
```

After this, the directory is renamed. If you were in `test/noss-1.10/`, you're now in `test/noss-2.03/`. Always check `pwd`.

Check what was created:

```
git log --oneline -5
git tag -n
```

You'll see: - A new commit on `upstream` branch: "New upstream version 2.03" - A merge commit on `master`: "Update upstream source from tag 'upstream/2.03'" - A new tag: `upstream/2.03`

5.4 Check What Changed

Before touching anything, see what upstream changed:

```
# What files changed?
git diff --stat upstream/1.10 upstream/2.03

# Did dependencies change?
git diff upstream/1.10 upstream/2.03 -- Makefile.PL

# Did a module get removed or added?
git diff --stat upstream/1.10 upstream/2.03 -- lib/
```

For our v2.03 update, we found: - New dependency: `Term::ANSIColor` (Perl core module — no extra package needed) - New runtime dep in `nossui`: `sensible-editor` from `sensible-utils` - `lib/WWW/Noss/Dir.pm` removed (tests removed too) - `lib/WWW/Noss/Util.pm` added

5.5 Update debian/ Files (If Needed)

Control: Add any new Build-Depends or Depends. For v2.03, we added `sensible-utils` to Depends.

Copyright: If upstream added files with new licenses, add stanzas. For v2.03, all new files were GPL-3.0+ (same as before), so no change.

Changelog: Add a new entry on top:

```
dch -v 2.03-1
```

This opens an editor. Write your entry. Your conventions: - all lowercase - `harinarayanmr@aol.com` - descriptive but short

Or write it manually with proper format:

```
noss (2.03-1) UNRELEASED; urgency=low

  * new upstream release.

-- hari narayan <harinarayanmr@aol.com>  Fri, 15 May 2026 04:02:02 +0530
```

Rules: Don't create files you don't need. Specifically: - Do NOT create `debian/install` (dh + dh_perl handle installation automatically) - Do NOT use AI-generated descriptions (use upstream's own text) - Every continuation line in Description must start with exactly one space

5.6 Build and Verify

```
sbuild -sA --no-clean-source --dist=unstable 2>&1 | tee build-2.03-1.log
```

Wait for completion. Check the summary at the end of the log:

```
tail -40 build-2.03-1.log
```

Look for:

```
Status: successful
Lintian: pass      # or at least no errors you can't explain
```

5.7 Check Lintian

```
lintian -i ../noss_2.03-1_amd64.changes
```

Read every error. Decide if you need to fix or can ignore:

This error...	...means	...fix?
E: unreleased-changes	Changelog says UNRELEASED	Ignore until upload
E: missing-depends-on-sensible-utils	Missing runtime dep	Fix — add to control
W: wrong-manual-section	Perl man page section	Upstream issue, add override
I: typo-in-manual-page	Spelling in man page	Upstream issue, report to author

5.8 Commit and Push

```
git add debian/control debian/changelog debian/watch
git commit -m "update to 2.03, fix deps"

git push --all
git push --tags
```

Always push both commits and tags. The tags are how gbp tracks upstream versions. Without tags, the next `gbp import-orig` will fail.

6. The Investigative Toolkit

This is the meta-skill: how to ask the system questions instead of guessing.

6.1 "What package owns this file?"

```
dpkg -S /usr/bin/sensible-editor
# Output: sensible-utils: /usr/bin/sensible-editor
```

This tells you which Debian package provides a given file. Use this when lintian says "missing-depends-on-sensible-utils" — verify it's real.

6.2 "Is this a valid Debian package name?"

```
apt-cache show sensible-utils
# Shows package info if it exists
apt-cache search sensible-utils
# Shows matching packages
```

Use this to verify package names before adding them to control.

6.3 "What Perl modules does this need?"

```
grep -A30 'PREREQ_PM' Makefile.PL
```

This shows the Perl modules the upstream project requires. Each `'Module::Name'` maps to Debian package `libmodule-name-perl`.

6.4 "What changed between upstream versions?"

```
# Summary of all file changes
git diff --stat upstream/1.10 upstream/2.03

# Specific file (e.g. dependency changes)
git diff upstream/1.10 upstream/2.03 -- Makefile.PL

# See if a file was added or removed
git diff --stat upstream/1.10 upstream/2.03 -- lib/
```

This is how you discover new deps, removed modules, or new test files.

6.5 "Did my build include my fix?"

```
grep -i 'sensible-utils' build-2.03-1.log
# Shows every time sensible-utils was referenced in the build
```

If it appears in the "Installed packages" section and in the "Depends" line of the built `.deb`, your fix worked.

6.6 "What's in my build log?"

```
grep '^E:|^W:' build-2.03-1.log      # lintian errors/warnings
grep 'Status:' build-2.03-1.log      # build status
grep 'Package-Time:' build-2.03-1.log # how long it took
```

6.7 "What does this lintian error mean?"

```
lintian -i --no-info # shows just errors with explanations
# OR
lintian-info -t tag-name # check a specific tag
```

6.8 "What does the git history look like?"

```
git log --oneline --all --graph
```

This shows the branching structure: master, upstream, pristine-tar, and tags. You'll see how `gbp import-orig` creates a branch structure:

```
*    ba98d58 update to 2.03, fix deps          ← your commit on master
*    8a8379a Update upstream source from tag 'upstream/2.03' ← gbp merge
|\
| * 94a2f99 New upstream version 2.03        ← upstream branch
|/
* 319eb8b fix debian/watch format            ← your commit on master
```

6.9 "What files does my .deb contain?"

```
dpkg -c ../noss_2.03-1_all.deb # list files in the .deb
# OR after installing:
dpkg -L noss
```

6.10 "What's the current state of my repo?"

```
git status          # uncommitted changes?
git tag -n          # what tags exist?
git branch -a       # what branches exist?
git remote -v       # what remote am I pushing to?
```

7. The Real Workflow (Abhijith's Sequence)

This is the exact sequence Abhijith taught you, in order:

1. File ITP bug on bugs.debian.org
2. Get salsa account approved
3. Install tools: devscripts, sbuild, git-buildpackage, quilt
4. Clone the salsa repo (SSH)
5. `gbp import-orig --verbose --pristine-tar ../noss-<ver>.tar.gz`
6. Edit debian/ files (control, copyright, changelog, watch)
7. `sbuild -sA --no-clean-source --dist=unstable`
8. `lintian -i ../noss_<ver>_amd64.changes` → fix issues → rebuild
9. `git push --all && git push --tags`
10. Sponsor uploads to Debian

For updates (most common):

1. `uscan --verbose --report` # check for new version
2. `uscan --download-version <ver>` # download tarball
3. `gbp import-orig --verbose --pristine-tar ../tarball`
4. `git diff upstream/OLD upstream/NEW` # check changes
5. Update debian/ files if needed
6. `sbuild -sA --no-clean-source --dist=unstable`
7. `lintian -i` → fix → rebuild if needed
8. `git push --all && git push --tags`

8. Real Mistakes Vault

Every mistake from your chat with Abhijith and our session. Don't repeat them.

Mistake 1: debian/install File

What happened: You created `debian/install` to manually install binaries because AI told you to. It caused:

```
cp: cannot create regular file 'debian/noss/usr/bin/noss': Permission denied
```

Why: For Perl packages, `dh` + `dh_perl` + `dh_install` handle everything automatically. `ExtUtils::MakeMaker` already knows where to install files. Adding a manual install file conflicts with the auto-installation.

Fix: Delete `debian/install`. Don't create it unless you have files that the build system won't install (e.g., upstream forgot to include a config file).

Lesson: "Now you know, don't trust Gemini or chatgpt too much" — Abhijith.

Mistake 2: AI-Generated Descriptions

What happened: You used AI to write the Description in control. It was too long, wrong tone, and didn't describe the package accurately.

Fix: Use upstream's own description. For `noss`, run `noss help` and use that text. Or use the README. The description should be factual, not marketing.

Mistake 3: Description Alignment

What happened: Lintian error because Description continuation lines weren't aligned properly.

Rule: In `debian/control`, the short description (first line) has no leading space. Every continuation line starts with **exactly one space**:

```
Description: short description here
             continuation line here
             another continuation line here
```

No tabs. No extra spaces. Wrong alignment → lintian error.

Mistake 4: .git Tracked in Source Package

What happened: The `.git` directory was accidentally included in the source package. This bloats the source and leaks git history.

Fix: Make sure `debian/source/format` contains:

```
3.0 (quilt)
```

The `3.0 (quilt)` format automatically excludes `.git` and other VCS directories. Abhijith pushed a fix for this.

Mistake 5: Tags Not Pushed

What happened: You ran `git push` but didn't push tags. Abhijith couldn't pull the upstream source.

Fix: Always push tags explicitly:

```
git push --tags
# OR
git push origin upstream/2.03
```

Mistake 6: Missing Dependencies

What happened: Multiple missing deps: - `dialog` — needed by `nossui` (TUI frontend) at runtime - `sensible-utils` — needed by `nossui v2.03` (uses `sensible-editor`) - `libdbd-sqlite3-perl` and `libparallel-forkmanager-perl` — initially missing from Build-Depends, causing `sbuid` to fail

How to avoid: Check `Makefile.PL` (`PREREQ_PM`), `bin/nossui` (for external commands like `dialog` , `sensible-editor`), and `README`.

Mistake 7: Lintian strictness

What happened: `sbuid` failed on lintian warnings, not just errors.

Why: Your `sbuid` config has `--fail-on error,warning` , which exits non-zero even for warnings. This is the Debian-standard config.

Fix: Either fix the warnings or adjust the config. For `wrong-manual-section` warnings, you can add a lintian override in `debian/source/lintian-overrides` .

Mistake 8: UNRELEASED vs unstable

What happened: Your changelog says `UNRELEASED` . Lintian flags this as an error: `E: noss changes: unreleased-changes` .

Why: `UNRELEASED` means "I'm working on this, don't upload it." It's correct for development. Change to `unstable` only when you're ready to release.

Mistake 9: sbuid Chroot Too Old

What happened: After ~7 days, `sbuid` warns:

```
Existing chroot tarball is too old (16.54 >= 7.00 days)
```

Fix: Update the chroot:

```
sbuid-update -udcar unstable
```

Or set persistent caching in `~/config/sbuild/config.pl` :

```
$unshare_mmdebstrap_keep_tarball = 1;
```

Mistake 10: --no-clean-source Needed

What happened: sbuid failed with:

```
E: Failed to clean source directory
I: use sbuid --no-clean-source to skip the cleanup
```

Why: sbuid tries to clean the source tree before building. If build deps aren't met on your host, the clean step fails.

Fix: Always use `--no-clean-source` :

```
sbuid -sA --no-clean-source --dist=unstable
```

Mistake 11: Watch File Wrong Format

What happened: `uscan` couldn't find new versions because the watch file used mixed v1 and v5 syntax.

Wrong (v1 syntax with Version: 5 header):

```
Version: 5
Source: https://codeberg.org/1-lsam/noss/tags
Matching-Pattern: .*/noss/archive/(\d[.\d]+)\.tar\.gz
```

Right (v4 format):

```
version=4
https://codeberg.org/1-lsam/noss/tags .*/noss/archive/(\d[.\d]+)\.tar\.gz
```


Mistake 12: HTTPS Remote Instead of SSH

What happened: Pushing to salsa failed:

```
fatal: could not read Username for 'https://salsa.debian.org'
```

Fix:

```
git remote set-url origin git@salsa.debian.org:debian/noss.git
```

9. Quick Reference (One-Page Cheat Sheet)

Package building

```
# Quick test build (no sign, no chroot)
dpkg-buildpackage -us -uc

# Clean-room build
sbuidl -sA --no-clean-source --dist=unstable

# Static analysis
lintian -i ../noss_<version>_amd64.changes
```

Upstream update cycle

```
# Check for new version
uscan --verbose --report

# Download
uscan --download-version <VER>
# OR if uscan fails:
curl -L -o ../noss-<VER>.tar.gz https://codeberg.org/1-lsam/noss/archive/<VER>.tar.gz

# Import
gbp import-orig --verbose --pristine-tar --upstream-version=<VER> ../noss-<VER>.tar.gz

# Check changes
git diff --stat upstream/<OLD> upstream/<NEW>
git diff upstream/<OLD> upstream/<NEW> -- Makefile.PL

# Build
sbuild -sA --no-clean-source --dist=unstable
```

Git/Salsa

```
# Push everything
git push --all
git push --tags

# Check state
git status
git tag -n
git branch -a
git log --oneline --all --graph
```

debian/ file validation

```
# Reformat consistently
wrap-and-sort

# Validate syntax
debputy
```

Investigation

```
# What package owns this file?
dpkg -S /path/to/file

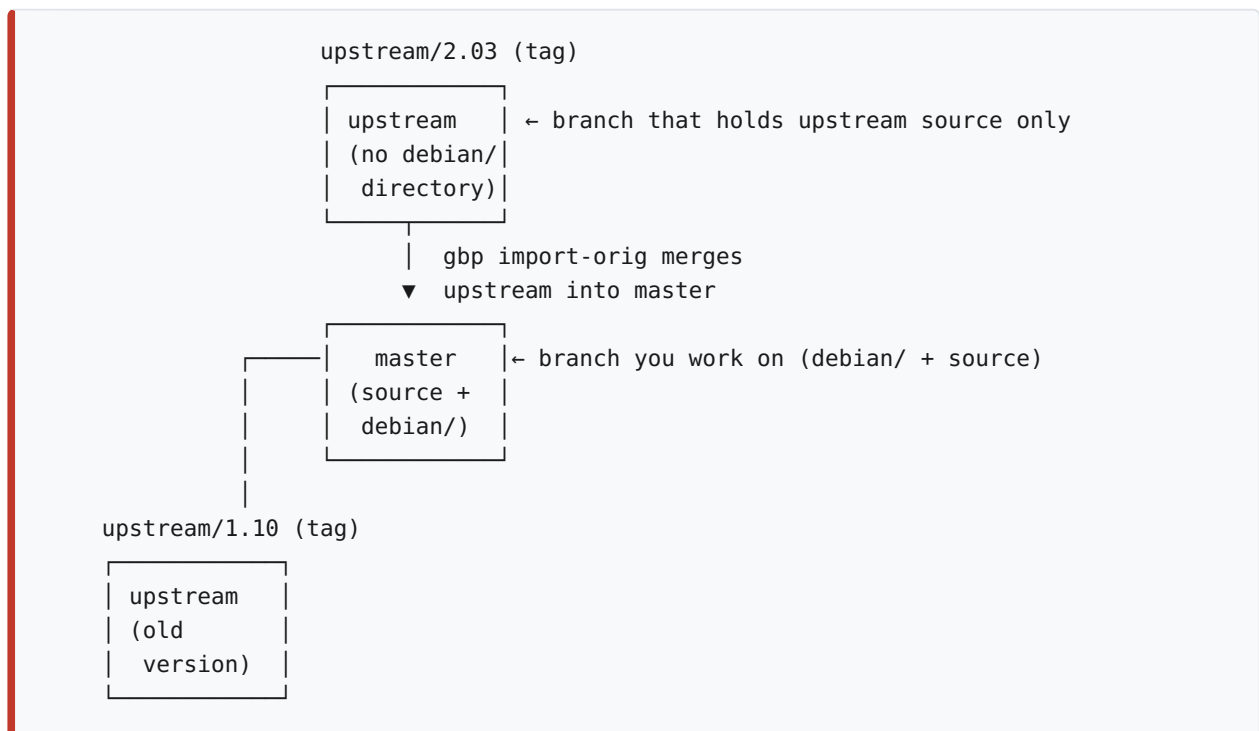
# Is this a real package?
apt-cache show <package-name>

# What's in the build log?
grep '^E:|^W:' build.log
grep 'Status:' build.log

# What does the .deb contain?
dpkg -c ../noss_<version>_all.deb
```

10. Understanding Git Branches in gbp

This is the most confusing part for newcomers. Here's what each branch/tag is:



- **master** : Where you work. Has both upstream source and `debian/` directory.
- **upstream** : Holds upstream source only (no `debian/`). `gbp import-orig` updates this branch.
- **pristine-tar** : Stores binary deltas so the exact orig tarball can be recreated from git.

- `upstream/<version>` **tags**: Pointers to specific upstream versions. `gbp import-orig` creates these. You must push them.

11. What Happens After You Push

1. **Salsa CI** runs automatically (if enabled via `debian/salsa-ci.yml`): - Builds your package - Runs lintian - Reports pass/fail on the salsa web page
2. **When you're ready to upload**, change `UNRELEASED` → `unstable` in changelog. Your sponsor runs `dput` to upload the `.changes` file.
3. **The Debian archive** runs its own checks (FTP masters review new packages, buildds build for all architectures).
4. **Your package appears** on `packages.debian.org` and `tracker.debian.org`. Users can `apt install noss`.
5. **You appear** on your DDPO page: `https://qa.debian.org/developer.php?login=harinarayanmr%40aol.com`
6. **When upstream releases v2.04**, you repeat chapter 5.

12. Final Words

Debian packaging is not magic. It's 5 files and 3 commands repeated in a loop. The skill is not in memorizing flags — it's in knowing *how to ask the system questions* when something goes wrong.

Every time you run a command and read its output, you learn one more thing. That's what I did in this session. That's what Abhijith taught you to do. That's all it takes.

Good luck, maintainer.